Matrix Free methods

Giorgio Negro

February 22, 2026

1 Introduction and Problem Formulation

1.1 Overview

This report documents an analysis comparing matrix-free (MF) and matrix-based (MB) implementations of finite element solvers for partial differential equations. This document focuses on scalability, memory efficiency, and computational performance in MPI-distributed computing environments.

1.2 Why Matrix Free methods?

Typical finite element solvers assemble a global stiffness matrix, representing the discretized PDE operator, thus requiring computation with large sparse matrices. This approach is by nature memory intensive and can lead to a severe underutilization of modern CPU capabilities. Matrix free methods compute the action of the operator without explicitly forming the matrix, reducing then the memory consumption and improving vectorization.

1.3 Problem Statement

The advection-diffusion-reaction problem as the following form:

$$-\nabla \cdot (\mu \nabla u) + \nabla \cdot (\boldsymbol{\beta} u) + \gamma u = f \quad \text{in } \Omega$$

with mixed boundary conditions on Γ_D and $\Gamma_N = \partial\Omega \setminus \Gamma_D$.

1.4 Implementation Choice

For the benchmark implementation, we chose a **divergence-free advection field** $\boldsymbol{\beta}(x, y) = (y, -x)$ with $\nabla \cdot \boldsymbol{\beta} = 0$. With a divergence-free field, the conservative form $\nabla \cdot (\boldsymbol{\beta} u)$ and non-conservative form $\boldsymbol{\beta} \cdot \nabla u$ are mathematically equivalent, allowing us to use the simpler non-conservative implementation and focus our effort on the performance analysis.

1.5 Model Problem

We solve an advection-diffusion-reaction (ADR) model on the unit square $\Omega = [0, 1]^2$:

$$-\nabla \cdot (\mu \nabla u) + \boldsymbol{\beta} \cdot \nabla u + \gamma u = f \quad \text{in } \Omega$$

with mixed boundary conditions:

$$u = g \quad \text{on } \Gamma_D \subset \partial\Omega$$
$$\nabla u \cdot \boldsymbol{n} = h \quad \text{on } \Gamma_N = \partial\Omega \setminus \Gamma_D$$

In this work:

- Dirichlet boundary: $\Gamma_D = \{x = 0\} \cup \{y = 0\}$
- Neumann boundary: $\Gamma_N = \{x = 1\} \cup \{y = 1\}$

1.6 Weak Formulation

Let V be the trial space with Dirichlet trace and V_0 the homogeneous test space. Find $u \in V$ such that for all $v \in V_0$:

$$a(u, v) = \ell(v)$$

where the bilinear and linear forms are:

$$a(u, v) = \int_\Omega [\mu \nabla u \cdot \nabla v + (\boldsymbol{\beta} \cdot \nabla u)v + \gamma uv]\, dx$$
$$\ell(v) = \int_\Omega f v\, dx + \int_{\Gamma_N} hv\, ds$$

1.7 Mathematical Formulation and Iterative Solver Integration

The transition from a matrix-based to a matrix-free framework is mathematically grounded in the requirements of iteratives methods, such as the right-preconditioned GMRES algorithm used in this analysis. These iterative solvers do not require explicit access to the individual entries A_{ij} of a global stiffness matrix. Instead, they view the linear system $Au = f$ through the lens of a linear operator, where the only necessary operation is the evaluation of the matrix-vector product (the "action" of the operator) $y = Av$ for a given search direction v. In the matrix free implementation the action is computed on the fly by evaluating the weak form of the integral on a per-element basis.

1.8 Discretization

We used a standard Galerkin finite element method with continuous Lagrange finite elements Q_p, $p \in \{2, 3, 4\}$ on an MPI-distributed mesh with distributed degrees of freedom, and a geometric multigrid hierarchy constructed from global refinements.

2 Solver Architectures

2.1 Common Solver methodology

Both implementations use the same solver strategy to ensure a fair comparison:

- **Solver**: Right-preconditioned GMRES
- **Preconditioner**: Geometric multigrid with Chebyshev smoothers

The multigrid configuration is kept consistent across both backends, with parameters as defined in the table Table 1.

Table 1: Default configuration for geometric multigrid

Parameter	Value
Chebyshev smoother degree	5
Smoothing range	15.0
Eigenvalue estimation iterations	10
Jacobi relaxation parameter ω	1.0
Coarse-level Chebyshev degree	12

3 Evaluations

The evaluation demonstrates and try to provide insights on the reason behind the performance gap between matrix-free and matrix-based implementations. **Testbed.** We evaluated both implementations on a single socket x86_64, 6 physical core Ryzen 5 2600 with boost disabled, using 32GB of RAM achieving up to 26.17 GB/s memory bandwidth.

3.1 Test Configuration

Unless otherwise specified, all experiments use the following configuration:

- **MPI ranks**: $n_p \in \{1, 2, 4, 6\}$
- **Polynomial degrees**: P2, P3, P4
- **Refinement levels**: Varied to achieve comparable problem sizes

3.2 Performance Metrics

Strong scaling:

$$
\begin{aligned}
T_{n_p} &= \text{runtime with } n_p \text{ ranks} \\
S_{n_p} &= \frac{T_1}{T_{n_p}} \quad \text{(speedup)} \\
E_{n_p} &= \frac{S_{n_p}}{n_p} \quad \text{(efficiency)}
\end{aligned}
$$

Gap decomposition:

$$
\begin{aligned}
\Delta_{\text{total}} &= T_{\text{MB}} - T_{\text{MF}} \\
\Delta_{\text{MPI}} &= T_{\text{MB}} \cdot f_{\text{MPI,MB}} - T_{\text{MF}} \cdot f_{\text{MPI,MF}} \\
r_{\text{MPI}} &= \frac{\Delta_{\text{MPI}}}{\Delta_{\text{total}}}
\end{aligned}
$$

3.3 Strong Scaling

3.3.1 Primary Strong Scaling Data

Results for highest common refinement levels ($n_p : 1 \to 6$):

Table 2: Strong scaling: P2 and P3 (refinement level 7 and 8)

Config	T_1 (s)	T_6 (s)	S_6	E_6 (%)	Refs
P2 MF	0.570	0.253	2.25	37.5	ref7
P2 MB	0.711	0.374	1.90	31.7	ref7
P3 MF	3.576	1.046	3.42	57.0	ref8
P3 MB	7.287	2.942	2.48	41.3	ref8

Table 3: Strong scaling: P4 (refinement level 8)

Config	T_1 (s)	T_6 (s)	S_6	E_6 (%)	Refs
P4 MF	5.702	1.624	3.51	58.5	ref8
P4 MB	15.442	5.448	2.83	47.2	ref8

3.3.2 Large problem strong scaling progression

Table 4: Complete P4 strong scaling across all rank counts (ref8, 5 repeats)

Solver	n_p	Runtime (s)	Speedup	Efficiency (%)	Std Dev (s)
Matrix-Free	1	5.702	–	100.0	0.270
Matrix-Free	2	3.121	1.827	91.4	0.036
Matrix-Free	4	1.831	3.115	77.9	0.056
Matrix-Free	6	1.624	3.511	58.5	0.222
Matrix-Based	1	15.442	–	100.0	0.707
Matrix-Based	2	9.437	1.636	81.8	0.399
Matrix-Based	4	6.102	2.531	63.3	0.248
Matrix-Based	6	5.448	2.834	47.2	0.318

3.3.3 Observations

Matrix-free consistently outperforms matrix-based, with the runtime advantage growing with both parallelism and polynomial degree. At $n_p = 6$, MF is 3.35× faster than MB for P4 (1.624 s vs 5.448 s), maintaining 58.5% parallel efficiency against MB's 47.2%. For P3 the gap is 2.81× (efficiency 57.0% vs 41.3%).

3.4 Weak Scaling Results

3.4.1 Kernel Weak Scaling

To isolate solver kernel performance from variable convergence behavior, we conducted fixed-iteration weak scaling experiments at refinement level 8 with polynomial degree P4. The global mesh was scaled as: $n_p = 1$ (1×1), $n_p = 2$ (2×1), $n_p = 4$ (2×2), $n_p = 6$ (3×2), maintaining DoFs/rank within ±0.11% across all ranks. Each configuration executed 20 fixed GMRES iterations amortized over 3 solves, timing reports geometric mean.

Weak scaling efficiency. Ideal weak scaling maintains constant time/iteration as work per rank remains constant. We compute work-normalized weak efficiency as:

$$E_{\text{weak}}(n_p) = \frac{T_1 \cdot (D_{n_p}/D_1)}{T_{n_p}}$$

where T_{n_p} is Krylov time at rank n_p and D_{n_p}/D_1 is the DoFs/rank ratio relative to baseline.

Table 5: Kernel weak scaling: P4 refinement 8 (fixed 20 iterations, Krylov phase)

n_p	Solver	Mesh	DoFs/rank	Time/iter (ms)	E_{weak}
1	MF	1×1	1.0000	443.3	1.000
2	MF	2×1	0.9995	481.7	0.920
4	MF	2×2	0.9990	576.6	0.769
6	MF	3×2	0.9989	704.2	0.629
1	MB	1×1	1.0000	682.3	1.000
2	MB	2×1	0.9995	883.1	0.773
4	MB	2×2	0.9990	1463.7	0.466
6	MB	3×2	0.9989	2071.4	0.329

Matrix-free maintains superior weak scaling across all ranks. At $n_p = 6$, MF achieves 62.9% weak efficiency compared to MB's 32.9%, a 1.91× advantage. In terms of time per iteration, MF grows only 1.59× from 1 to 6 ranks, whereas MB grows 3.04×. While both methods exhibit non-ideal scaling MF degrades more gracefully across the full range of parallelism.

3.5 Phase Analysis

3.5.1 Top-Level Time Composition at Refinement 8 ($n_p = 6$)

Table 6: Phase shares of total runtime (%) at ref8, $n_p = 6$

Config	Setup / Total	Assembly / Total	Solve / Total
P3 MF	22.43	1.47	76.12
P3 MB	18.90	19.32	61.82
P4 MF	20.93	1.17	77.90
P4 MB	15.24	26.67	58.11

3.5.2 Solve Internal Composition (as an % of Solve)

Table 7: Solve-phase internal breakdown (%) at ref8, $n_p = 6$

Config	Preconditioner / Solve	Krylov / Solve	Other / Solve
P3 MF	5.40	94.31	0.29
P3 MB	19.76	79.90	0.33
P4 MF	3.74	96.01	0.25
P4 MB	15.94	83.85	0.21

At high work (ref8), most wall time is spent in Solve for all cases. MB spends a comparatively larger fraction of total runtime to Assembly and Preconditioner.

3.6 Memory Consumption Analysis

Peak resident memory (VmHWM), measured per rank (max across ranks):

Table 8: Peak memory footprint per rank: P3, ref8

n_p	MF HWM (MiB)	MB HWM (MiB)	Ratio MB/MF
1	530.6	1010.8	1.91×
2	427.5	713.6	1.67×
4	374.0	519.6	1.39×
6	357.1	459.6	1.29×

Table 9: Peak memory footprint per rank (HWM max): P4, ref8

n_p	MF HWM (MiB)	MB HWM (MiB)	Ratio MB/MF
1	661.4	1906.9	2.88×
2	487.4	1197.6	2.46×
4	410.2	774.4	1.89×
6	381.3	641.0	1.68×

Both degrees show the same trend: the MB/MF RSS ratio shrinks as n_p increases, since the global sparse matrix is distributed across more ranks and each rank holds a smaller local portion while per rank overhead remain constant. However, the absolute MB footprint per rank remains measurably higher than MF for all ranks, 2.88× at $n_p = 1$ for P4 where MB requires 1906 MB against MF's 661 MB and even at $n_p = 6$ the gap persists (641.0 vs 381.3 MB for P4).

4 MPI Communication Analysis

4.1 Method and Definitions

For each configuration, we define:

$$\begin{aligned} T_{\text{MPI}} &= T_{\text{coll}} + T_{\text{p2p}} + T_{\text{wait}}, \\ \Delta_{\text{total}} &= T_{\text{MB}} - T_{\text{MF}}, \\ \Delta_{\text{MPI}} &= T_{\text{MPI,MB}} - T_{\text{MPI,MF}}, \\ r_{\text{MPI}} &= \frac{\Delta_{\text{MPI}}}{\Delta_{\text{total}}}. \end{aligned}$$

The data was collected using a custom MPI tracer

4.2 MPI Scaling Behavior, Refinement 8

Table 10: MPI time evolution with increasing parallelism (P3, ref=8)

Config	n_p	T_{total} (s)	T_{MPI} (s)	**MPI %**
P3 MF	2	2.066	0.115	5.58
	4	1.195	0.149	12.46
	6	0.984	0.194	19.75
P3 MB	2	4.189	0.226	5.39
	4	2.822	0.376	13.34
	6	2.384	0.351	14.73

Table 11: MPI time evolution with increasing parallelism (P4, ref=8)

Config	n_p	T_{total} (s)	T_{MPI} (s)	**MPI %**
P4 MF	2	3.494	0.146	4.18
	4	1.941	0.203	10.45
	6	1.540	0.232	15.08
P4 MB	2	9.325	0.338	3.62
	4	6.154	0.536	8.71
	6	5.307	0.524	9.88

MF consistently exhibits a higher MPI percentage but a lower absolute MPI time compared to MB (e.g. at $n_p = 6$, MF records 19.75% (0.194s) compared to MB's 14.73% (0.351s) for P3, and 15.08% (0.232s) compared to 9.88% (0.524s) for P4. MB's MPI time initially increases and then saturates (e.g., for P4: 0.338s → 0.536s → 0.524s at $n_p = 6$), whereas MF's MPI time grows steadily and monotonically (e.g., for P3: 0.115s → 0.149s → 0.194s). MB's MPI time saturation is in line with the observed parallel efficiency degradation.

4.3 MPI Time Decomposition

Table 12: MPI breakdown at $n_p = 6$ (P3, ref=8)

Backend	Allreduce (s)	Barrier (s)	Other Coll. (s)	P2P+Wait (s)	Total (s)
MF	0.079	0.011	0.009	0.095	0.194
MB	0.173	0.077	0.046	0.055	0.351

Table 13: MPI breakdown at $n_p = 6$ (P4, ref=8)

Backend	Allreduce (s)	Barrier (s)	Other Coll. (s)	P2P+Wait (s)	Total (s)
MF	0.080	0.011	0.008	0.133	0.232
MB	0.266	0.131	0.063	0.064	0.524

Category definitions:

- **Allreduce**: `MPI_Allreduce` only (dominant collective)
- **Barrier**: `MPI_Barrier` only (explicit global synchronization)
- **Other Coll.**: `MPI_Reduce_scatter`, `MPI_Allgather`, `MPI_Bcast`, `MPI_Exscan`, `MPI_Gatherv`
- **P2P+Wait**: `MPI_Isend`, `MPI_Irecv`, `MPI_Rsend` (initiation) *plus* `MPI_Waitall`, `MPI_Waitsome`, `MPI_Test` (completion) — grouped together since wait time is time blocked on outstanding P2P requests, not an independent communication phase

The profiling reveals distinct communication patterns between the two backends. MB relies heavily on barriers, consuming 22–25% of its MPI time compared to only 5–6% for MF, this is due to MB's synchronous blocking communication pattern and memory-bound operations that cause load imbalance, leading to ranks reaching barriers less uniformly. MF instead favors non-blocking communication: its P2P+Wait column :accounts for 49–57% of total MPI time, reflecting asynchronous message passing where initiation and completion are decoupled. By contrast, MB's P2P+Wait is only 12–16% of its MPI total, consistent with its reliance on synchronous collective calls rather than non-blocking P2P. Allreduce operations is significant on for both backends (40–51% of MPI time), though MB spends 2–3× more in absolute terms.

The following tables show how these categories evolve with increasing parallelism:

Table 14: MPI time breakdown by category (P3, ref=8)

Config	T_{allred} (s)	T_{barr} (s)	T_{ocoll} (s)	$T_{\text{p2p+w}}$ (s)	T_{MPI} (s)
P3 MF $n_p = 2$	0.046	0.003	0.003	0.063	0.115
P3 MF $n_p = 4$	0.069	0.006	0.007	0.067	0.149
P3 MF $n_p = 6$	0.079	0.011	0.009	0.095	0.194
P3 MB $n_p = 2$	0.131	0.046	0.026	0.023	0.226
P3 MB $n_p = 4$	0.220	0.079	0.036	0.041	0.376
P3 MB $n_p = 6$	0.173	0.077	0.046	0.055	0.351

Table 15: MPI time breakdown by category (P4, ref=8)

Config	T_{allred} (s)	T_{barr} (s)	T_{ocoll} (s)	$T_{\text{p2p+w}}$ (s)	T_{MPI} (s)
P4 MF $n_p = 2$	0.054	0.003	0.003	0.086	0.146
P4 MF $n_p = 4$	0.084	0.006	0.005	0.108	0.203
P4 MF $n_p = 6$	0.080	0.011	0.008	0.133	0.232
P4 MB $n_p = 2$	0.193	0.074	0.044	0.027	0.338
P4 MB $n_p = 4$	0.272	0.166	0.052	0.047	0.536
P4 MB $n_p = 6$	0.266	0.131	0.063	0.064	0.524

MF's P2P+Wait grows steadily with n_p (more neighbours, more messages), while its collective time grows sublinearly. MB's collectives dominate and saturate at $n_p = 6$ in line with the memory-bandwidth plateau observed in Section 5.4, whereas MB's P2P+Wait remains negligible throughout.

5 Microarchitecture Performance

5.1 Cache and Memory Hierarchy

IPC, and cache miss counters for P4, $n_p = 6$, ref8 (from `perf` profiling):

Table 16: Cache and IPC metrics (end-to-end)

Backend	IPC	Cache Miss (%)	L1 Miss (%)	MPKI
MF	0.894	34.88	13.69	51.56
MB	1.134	30.09	8.97	23.22

Table 17: Cache and IPC metrics (fixed 20 iterations)

Backend	IPC	Cache Miss (%)	L1 Miss (%)	MPKI
MF	0.927	33.40	10.33	41.29
MB	1.054	25.26	8.22	17.92

The cache miss rates does not show a clear pattern that would explain the performance gap, notably MB has a lower cache miss rate and a higher IPC.

5.2 CPU Pipeline Analysis

Extended pipeline-oriented counters (P4, $n_p = 6$, ref8):

Table 18: CPU pipeline metrics (end-to-end)

Backend	IPC	FE Stall (%)	BE Stall (%)	uops/inst
MF	1.072	2.08	33.95	1.115
MB	1.373	2.23	40.00	0.942

Table 19: CPU pipeline metrics (fixed 20 iterations)

Backend	IPC	FE Stall (%)	BE Stall (%)	uops/inst
MF	1.120	2.05	36.85	1.202
MB	1.226	2.30	44.38	0.929

Where:

- **FE Stall**: Frontend stalled cycles / total cycles
- **BE Stall**: Backend stalled cycles / total cycles
- **uops/inst**: Retired micro-ops per instruction

Both backends are backend-limited (as expected from HPC workloads), with MB showing a higher backend stall and a lower micro-op per instruction ratio as opposed to the higher IPC. This shows that while MF IPC is lower the number of micro-ops per instruction is higher, which is consistent with the higher FMA usage observed in the next section.

5.3 SIMD and Vectorization Analysis

Direct measurement of vector instruction usage (P4, ref8):

Table 20: SIMD instruction metrics ($n_p = 1$, end-to-end)

Backend	SSE/AVX per kinst	FP ops per kinst	DP-FMA share (%)
MF	365	473	46.9
MB	367	159	1.4

Table 21: SIMD instruction metrics ($n_p = 6$, end-to-end)

Backend	SSE/AVX per kinst	FP ops per kinst	DP-FMA share (%)
MF	176	218	47.1
MB	254	107	1.4

Both backends use a similar share of SIMD instructions MF achieve much higher density of floating-point operations, and a dramatically higher share of FMA instructions (over fma+mul+add) compared to MB.

The implementation patterns directly explain the FMA disparity:

Matrix-Free uses vector-friendly combined operations:

```
phi.submit_gradient(mu * gradient, q);
phi.submit_value(beta * gradient + gamma * value, q);
```

The expression `beta * gradient + gamma * value` allows the compiler to use FMA instructions, increasing the float throughput.

Matrix-Based accumulates element matrix entries separately:

```
cell_matrix(i,j) += (mu * shape_grad(i,q) * shape_grad(j,q)
                   + beta * shape_grad(j,q) * shape_value(i,q)
                   + gamma * shape_value(i,q) * shape_value(j,q))
                   * JxW(q);
```

The `+=` accumulation and separate term evaluation inhibit FMA fusion. Each multiply-add becomes two separate operations (multiply, then add to accumulator).

5.4 Memory Bandwidth Characterization

To assess whether DRAM bandwidth limits scalability, we used two complementary measurements:

- STREAM Triad as platform reference peak
- application PMU traffic (`perf` metric `nps1_die_to_dram`) as a DRAM-pressure indicator

STREAM reference peak. STREAM was compiled with OpenMP and run with thread pinning. The best geomean Triad rate was 26.17 GiB/s at 4 threads, which is the reference peak for this test environment.

PMU-to-STREAM calibration. The PMU metric and STREAM are different scales. On this host, simultaneous runs gave:

$$k_{\text{PMU}\to\text{STREAM}} \approx 2.60, \qquad B_{\text{STREAM-eq}} = \frac{B_{\text{PMU}}}{2.60}.$$

Application bandwidth utilization: We measured DRAM bandwidth across all rank counts for P4 refinement 8 in end-to-end strong scaling mode (geometric mean over 5 repeats):

Table 22: DRAM bandwidth usage: P4 ref8 (fixed-iters=20, amortize-solves=3, STREAM-equivalent, geomean)

Solver	n_p	STREAM-eq BW (GiB/s)	% of STREAM Peak	Total time (s)
Matrix-Free	1	4.83	18.4	9.269
Matrix-Free	2	7.54	28.8	5.521
Matrix-Free	4	9.19	35.1	3.073
Matrix-Free	6	9.82	37.5	2.473
Matrix-Based	1	11.31	43.2	18.748
Matrix-Based	2	17.68	67.5	11.202
Matrix-Based	4	21.14	80.8	8.074
Matrix-Based	6	21.17	80.9	7.711

The data reveal a clear contrast in memory pressure. Matrix-based (MB) reach to high bandwidth utilization by $n_p = 4$ (80.8% of STREAM peak) and then plateaus at $n_p = 6$ (80.9%), indicating that additional ranks provide little extra memory-throughput headroom. Matrix-free (MF), by contrast, remains substantially lower across all ranks (18.4% to 37.5%) and increases more gradually with n_p, consistent with a less bandwidth-constrained execution path.

This is consistent with the strong-kernel timing trend: MB improves only marginally from $n_p = 4$ to $n_p = 6$ (8.074 s $\to$ 7.711 s, 4.5%), while MF still gains more (3.073 s $\to$ 2.473 s, 19.5%). Thus, MB appears closer to a memory-bandwidth ceiling in this regime, whereas MF retains more scaling headroom.

6 Notes on Implementation Complexity

The matrix-free and matrix-based solvers differ significantly in implementation complexity. The matrix-free solver avoids explicit matrix assembly, requiring implementation of operator application kernels, on-the-fly evaluation of diffusion, advection, and reaction coefficients, explicit quadrature-based evaluation paths. The matrix-based solver assembles sparse matrices at all

multigrid levels and relies on standard sparse linear algebra operations, making the operator path conceptually simpler and easier to inspect.

In this project, both approaches use the same preconditioning family (geometric multigrid with Chebyshev smoothing) for a fair comparison. However, matrix-based formulations can in easily integrate a broader class standard of algebraic preconditioners (e.g., ILU/AMG variants) more directly, while these are harder to apply in a matrix-free solver because they require explicit matrix entries.

7 Scope Limitations

- **Hardware scale**: Limited to $n_p \leq 6$ due to hardware constraints
- **Problem class**: Single PDE type (ADR) on regular domains
- **Architecture**: Single compute node, shared-memory effects present
- **Performance counter caveats**:
  - Stall counters are attribution indicators, not additive decompositions
  - Event availability is architecture-specific (AMD Zen+)
  - Results serve as comparative indicators, not absolute characterization

8 Future Directions

1. **General advection fields**: Extend to non-divergence-free advection ($\nabla \cdot \boldsymbol{\beta} \neq 0$) requiring conservative form implementation
2. **Extended scaling studies**: Test beyond $n_p = 6$ to identify scalability limits
3. **GPU acceleration**: Investigate matrix-free performance on GPU architectures